Coding Challenge: Weather

Below are the details needed to construct a weather based app where users can look up weather for a city.

Public API
Create a free account at openweathermap.org. Just takes a few minutes. Full documentation for the service below is on their site, be sure to take a few minutes to understand it.

//

https://api.openweathermap.org/data/2.5/weather?lat=44.34&lon=10.99&appid={API key}

Built-in geocoding

Please use **Geocoder API** if you need automatic convert city names and zip-codes to geo coordinates and the other way around.

Please note that API requests by city name, zip-codes and city id have been deprecated. Although they are still available for use, bug fixing and updates are no longer available for this functionality.

Built-in API request by city name

You can call by city name or city name, state code and country code. Please note that searching by states available only for the USA locations.

API call

https://api.openweathermap.org/data/2.5/weather?q={city name}&appid={API key}

https://api.openweathermap.org/data/2.5/weather?q={city name},{country code}&appid={API key}

https://api.openweathermap.org/data/2.5/weather?q={city name},{state code},{country code}&appid={API key}

You will also need icons from here:

http://openweathermap.org/weather-conditions

//

Requirements

These requirements are rather high-level and vague. If there are details I have omitted, it is because I will be happy with any of a wide variety of solutions. Don't worry about finding "the" solution.

1. Create a browser or native-app-based application to serve as a basic weather app.
2. Search Screen

    1. Allow customers to enter a US city
    2. Call the openweathermap.org API and display the information you think a user would be interested in seeing. Be sure to has the app download and display a weather icon.
    3. Have image cache if needed

3. Auto-load the last city searched upon app launch.
4. Ask the User for location access, If the User gives permission to access the location, then retrieve weather data by default

In order to prevent you from running down rabbit holes that are less important to us, try to prioritize the following:

What is Important

- Proper function – requirements met.
- Well-constructed, easy-to-follow, commented code (especially comment hacks or workarounds made in the interest of expediency (i.e. // given more time I would prefer to wrap this in a blah blah blah pattern blah blah)).
- Proper separation of concerns and best-practice coding patterns.
- Defensive code that graciously handles unexpected edge cases.

What is Less Important

- UI design – generally, design is handled by a dedicated team in our group.
- Demonstrating technologies or techniques you are not already familiar with (for example, if you aren't comfortable building a single-page app, please don't feel you need to learn how for this).

iOS:

Must Have:

- **Architecture**: Use MVVM-C (Model-View-ViewModel-Coordinator).
- **UI Frameworks**: Utilize both UIKit and SwiftUI; ***do not use storyboards***.
- **Dependency Injection**: Implement dependency injection for managing dependencies.
- **Orientation and Size Classes**: Support both landscape and portrait orientations; adapt UI for different size classes.
- **Third-Party Libraries**: Do not use any third-party libraries; rely solely on native Swift and Apple frameworks.
- **Testing**: Write unit tests for ViewModel and Model layers;
- **Error Handling**: Implement robust error handling with user-friendly messages.

Nice to have :

- **UITest case:** write UI tests for user interface and interactions.
- **Performance**: Optimize the application for smooth and responsive user interactions.

- **Accessibility**: Ensure the application is accessible, supporting features like VoiceOver.
- **Localization**: Prepare the application for localization to support multiple languages.

Android:

- Make sure you are correctly handing any necessary permissions.
- Must have technologies as follows

Type	Must have
Coding Language	Kotlin (To demonstrate the use of Java, we'd rather you use a combination of Java and Kotlin)
Architecture Pattern	MVVM
Network library	Retrofit
Unit test	Junit

- Nice to have technologies as follows

Type	Nice to have
Network - Concurrency	Rx Java , Kotlin Coroutines
Unit test	Espresso or Mockito
UI	Jetpack Compose
Dependency Injection	Dagger 2 or Hilt or Koin
Navigation	Jetpack navigation

As mentioned, you are not expected to function in a vacuum. Use all the online resources you can find, and please do contact us with questions or for interim feedback if you desire.